



Module 01 — แนวคิด Data Engineering และสถาปัตยกรรมข้อมูลยุคใหม่

ผู้สอน: **พศ.ดร. เอกรัฐ รัชกาญจน์**

คณะสถิติประยุกต์ · สถาบันบัณฑิตพัฒนบริหารศาสตร์ (นิด้า)

เนื้อหาใน Module นี้

1. บทบาทของ Data Engineering ในองค์กร
2. โครงสร้างสถาปัตยกรรมข้อมูล (Data Warehouse, Data Lake, Lakehouse)
3. แนวคิด Batch Processing และ Stream Processing
4. การออกแบบ Workflow สำหรับการจัดการข้อมูล (ETL / ELT)

01

บทบาทของ Data Engineering ในองค์กร

Data Engineering คืออะไร

- สาขาวิชาที่เน้นการออกแบบ สร้าง และบำรุงรักษาระบบ สำหรับรวบรวม จัดเก็บ และประมวลผลข้อมูล
- ทำหน้าที่เป็นสะพานเชื่อมระหว่าง **Raw Data** กับ **Data-Driven Decision Making**
- ครอบคลุมตั้งแต่การ ingest ข้อมูลจากหลายแหล่ง ไปจนถึงการจัดเตรียมข้อมูลพร้อมใช้งาน

Data Engineering vs Data Science

	Data Engineering	Data Science
เป้าหมาย	สร้าง pipeline และ infrastructure	วิเคราะห์และสร้างโมเดล
ผลลัพธ์	ข้อมูลที่ clean, reliable, พร้อมใช้	Insight, prediction, dashboard
เครื่องมือ	Spark, Airflow, Kafka, SQL, ETL	Python, MLlib, Jupyter, TensorFlow
ทักษะหลัก	System design, programming, SQL	Statistics, ML, visualization

"Data Engineering คือ การสร้างรากฐานให้ Data Science สามารถทำงานได้"

บทบาทของ Data Engineering ในองค์กร

หน้าที่	คำอธิบาย
Data Ingestion	ออกแบบและพัฒนารูปแบบการรวบรวมข้อมูลจากหลายแหล่ง
Data Pipeline	ออกแบบและพัฒนารูปแบบการทำงานแบบเป็นลำดับขั้นตอน
Data Infrastructure	จัดการ Data Warehouse / Data Lake / Lakehouse
Data Quality	ตรวจสอบความถูกต้อง ความสมบูรณ์ และความสม่ำเสมอของข้อมูล
Monitoring	ติดตามการทำงานของ pipeline และประสิทธิภาพของระบบ
Collaboration	ทำงานร่วมกับ Data Scientist, Analyst, และ Stakeholder

ความท้าทายของ Data Engineering

- **Data Volume** — ข้อมูลมีปริมาณมหาศาล (Terabytes → Petabytes)
- **Data Variety** — ข้อมูลมีหลากหลายรูปแบบ (structured, semi-structured, unstructured)
- **Data Velocity** — ข้อมูลเข้ามาด้วยความเร็วสูง (real-time / near real-time)
- **Data Veracity** — คุณภาพของข้อมูลไม่แน่นอน ต้องมีการ cleansing และ validation
- **Scalability** — ระบบต้องรองรับการเติบโตของข้อมูลในอนาคต

ตัวอย่าง Scenario

ก่อนมี Data Engineering

- ข้อมูลถูกดึงมาจากหลายแผนกแบบไม่เป็นระบบ
- ใช้เวลานาน (ชั่วโมง/วัน) ในการรวมข้อมูล
- ข้อมูลมีโอกาสขัดแย้งกันค่อนข้างสูง
- ข้อมูลอัปเดตแบบ manual

หลังจากมี Data Engineering

- Single source of truth
- ลดเวลาในการรวบรวมข้อมูล
- มีวิธีการตรวจสอบที่มีประสิทธิภาพ
- ข้อมูลอัปเดตอัตโนมัติทุกวัน

02

โครงสร้างสถาปัตยกรรมข้อมูล (Data Warehouse, Data Lake, Lakehouse)

วิวัฒนาการสถาปัตยกรรมข้อมูล

ยุค 1990s —> Data Warehouse (โครงสร้างตายตัว)

ยุค 2010s —> Data Lake (ยืดหยุ่น เก็บทุกอย่าง)

ยุค 2020s —> Lakehouse (รวมข้อดีของทั้งสอง)

Data Warehouse

- **แนวคิด:** (Schema-on-Write) เก็บข้อมูลที่ผ่านการจัดโครงสร้างแล้ว
- **ลักษณะ:** ข้อมูลถูก transform ก่อนนำเข้า เหมาะสำหรับ BI และ Reporting
- **เทคโนโลยี:** Snowflake, Google BigQuery, Amazon Redshift, Teradata, etc.

ข้อดี

- Query เร็ว เพราะข้อมูลมีการจัดระเบียบแล้ว
- Data governance และ quality สูง
- เหมาะกับธุรกิจที่มีโครงสร้างข้อมูลแน่นอน

ข้อเสีย

- ยืดหยุ่นต่ำ เปลี่ยน schema ยาก
- ต้นทุนสูงในการจัดเก็บและประมวลผล

Data Lake

- **แนวคิด:** (Schema-on-Read) เก็บข้อมูล Raw Format ตามเดิม
- **ลักษณะ:** เก็บทั้ง structured, semi-structured, unstructured
- **เทคโนโลยี:** Amazon S3, Azure Data Lake, Google Cloud Storage, HDFS

ข้อดี

- ยืดหยุ่นสูง เก็บข้อมูลทุกรูปแบบ
- ต้นทุนต่ำ (object storage)
- รองรับ advanced analytics (ML, NLP, etc.)

ข้อเสีย

- อาจกลายเป็น **Data Swamp** (ข้อมูลที่ไม่เป็นระเบียบ) ถ้าไม่มี governance
- Query performance ต่ำกว่า Data Warehouse และ ต้องการ skill การจัดการ data ค่อนข้างสูง

Data Lake — Data Swamp Problem

ปัญหา	สาเหตุ	ผลกระทบ
ไม่มีข้อมูล Meta	ไม่มี catalog (สารบัญข้อมูล) หรือ schema	หาข้อมูลไม่เจอ
คุณภาพข้อมูลต่ำ	ไม่มี validation	วิเคราะห์ผิดพลาด
ข้อมูลซ้ำซ้อน	ไม่มี deduplicate	เปลืองพื้นที่
ไม่มี access control	ใครก็เข้าถึงได้	ความปลอดภัยต่ำ

Data Lake ≠ Data Swamp — ต้องมี governance, catalog, และ data quality

Lakehouse

- **แนวคิด:** รวมข้อดีของ Data Warehouse และ Data Lake
- **ลักษณะ:** เก็บ raw data ราคาถูก, มี ACID transaction, และ query เร็ว
- **เทคโนโลยี:** Databricks (Delta Lake), Apache Iceberg, Apache Hudi

จุดเด่น

- **ACID transactions** สำหรับ data lake
- **Schema enforcement** และ evolution
- **Time travel** — ย้อนดูประวัติข้อมูล
- **BI-ready** — query เร็วเท่า warehouse

ACID transactions

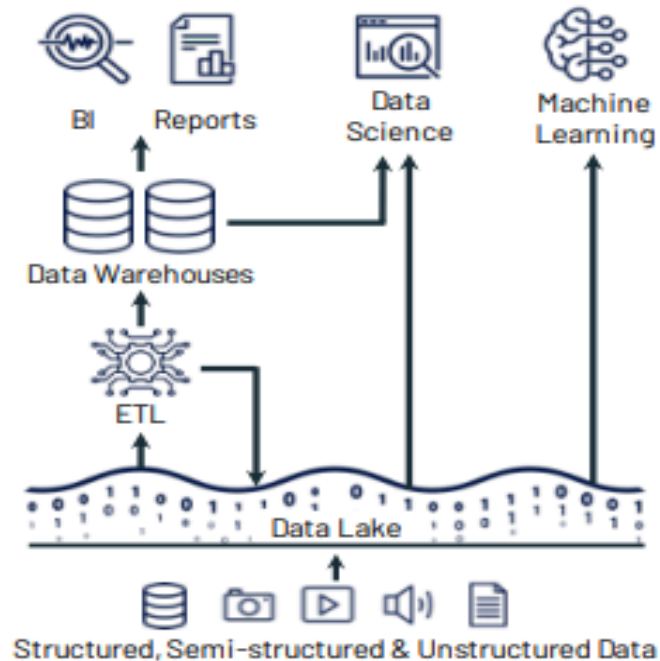
คือ ชุดของการดำเนินการ (เช่น การอ่านหรือเขียนข้อมูล) ในระบบฐานข้อมูล ที่รับประกันความถูกต้อง ความน่าเชื่อถือ และความปลอดภัยของข้อมูล โดยมีหลักการสำคัญ 4 ประการประกอบด้วย

1. **Atomicity (ความเป็นอะตอม)**: การทำงานแบบ "ทั้งหมดหรือไม่มีเลย" หากขั้นตอนใดขั้นตอนหนึ่งล้มเหลว ระบบจะยกเลิกการทำงานทั้งหมดและคืนค่ากลับสู่สถานะเดิม (เช่น หากโอนเงินไม่สำเร็จ เงินจะไม่ถูกหักจากบัญชีต้นทางและไม่หายไป)
2. **Consistency (ความสอดคล้อง)**: ข้อมูลที่เปลี่ยนแปลงต้องถูกต้องตามกฎหมายและเงื่อนไขของฐานข้อมูลเสมอ ไม่มีข้อมูลที่ขัดแย้งกันเกิดขึ้น
3. **Isolation (การแยกออกจากกัน)**: หากมีการทำธุรกรรมพร้อมกันหลายรายการ ระบบจะแยกการทำงานออกจากกันเพื่อให้มั่นใจว่าแต่ละธุรกรรมจะไม่ยุ่งเกี่ยวหรือก้าวก่ายกันจนเกิดข้อผิดพลาด
4. **Durability (ความคงทน)**: เมื่อธุรกรรมเสร็จสมบูรณ์ (Commit) ข้อมูลจะถูกบันทึกอย่างถาวร แม้ระบบจะเกิดปัญหาไฟดับหรือเซิร์ฟเวอร์ล่ม

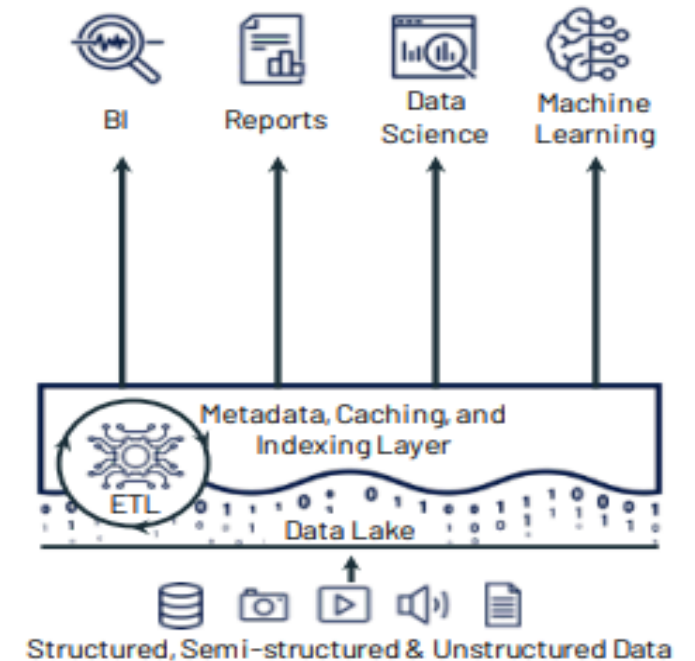
เปรียบเทียบสถาปัตยกรรม (1/2)



(a) First-generation platforms.



(b) Current two-tier architectures.



(c) Lakehouse platforms.

Armbrust M. et al, "Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics." In Proceedings of CIDR, vol. 8, no. 1, p. 28. sn, 2021.

เปรียบเทียบสถาปัตยกรรม (2/2)

คุณสมบัติ	Data Warehouse	Data Lake	Lakehouse
รูปแบบข้อมูล	Structured	All types	All types
Schema	Write-on-schema	Read-on-schema	Both
ACID	Yes	No	Yes
ต้นทุน	สูง	ต่ำ	ปานกลาง
Query speed	เร็วมาก	ช้า	เร็ว
ML/AI support	ต่ำ	สูง	สูง
Governance	สูง	ต่ำ-ปานกลาง	สูง

03

แนวคิด Batch Processing และ Stream Processing

Batch Processing

- ประมวลผลข้อมูลเป็นชุด (batch) ในช่วงเวลาที่กำหนด เช่น ทุกเที่ยงคืน ทุกชั่วโมง
- เหมาะกับข้อมูลปริมาณมาก แต่ไม่ต้องการ real-time

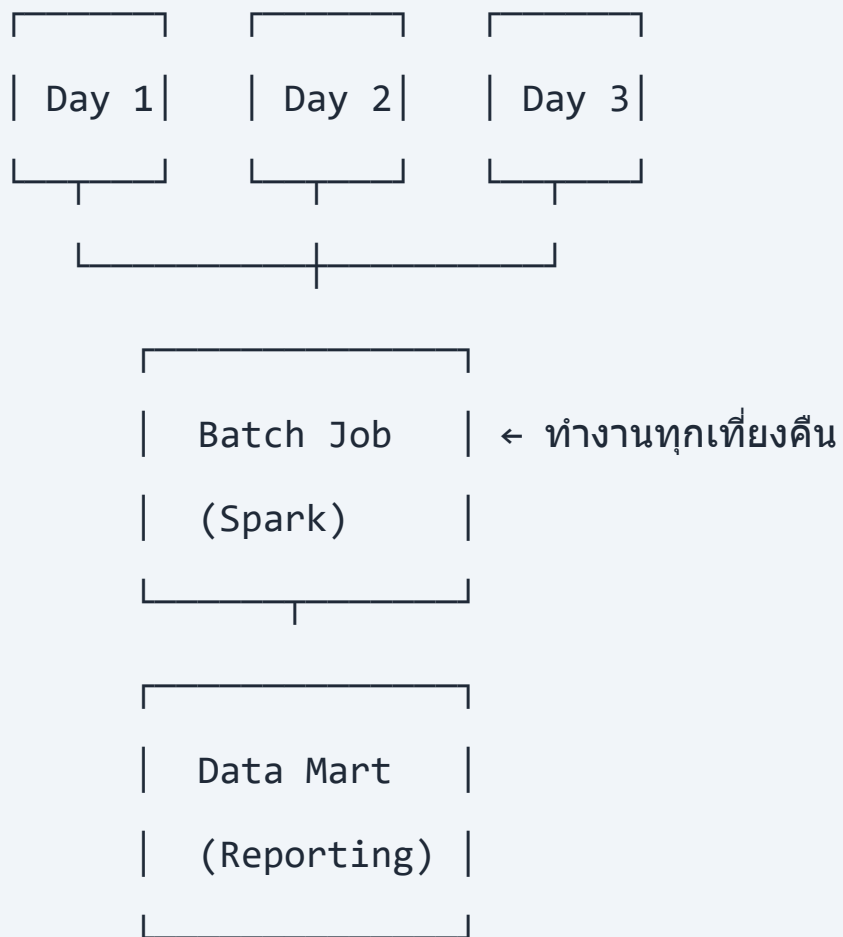
ลักษณะ

- มี **start time** และ **end time** ที่แน่นอน
- เหมาะกับงาน ETL, รายงานสรุปประจำวัน/เดือน
- **เทคโนโลยี:** Apache Spark, Hadoop, MapReduce, Airflow

ข้อดี

- จัดการข้อมูลปริมาณมากได้ดี
- ง่ายต่อการ error handling และ retry

Batch Processing — ตัวอย่าง



-- ตัวอย่าง: รายงานยอดขายประจำวัน

```
SELECT date, SUM(amount) as daily_revenue
FROM sales
WHERE date = CURRENT_DATE - 1
GROUP BY date;
```

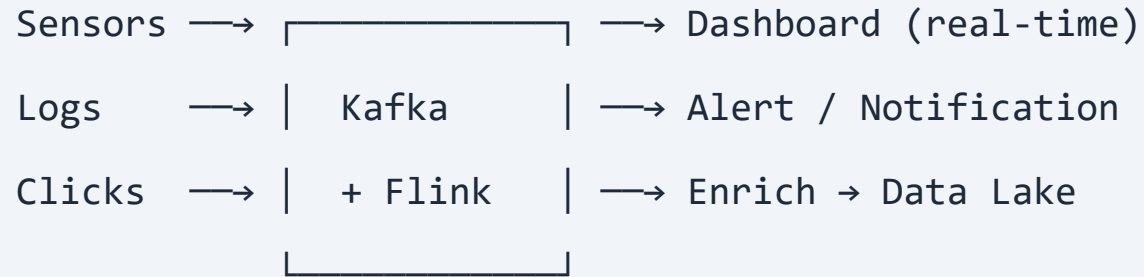
Stream Processing

- **ประมวลผลข้อมูลแบบต่อเนื่องทันทีที่ได้รับ** - เช่น ข้อมูลจาก sensor, clickstream, log
- เหมาะกับงานที่ต้องการผลลัพธ์แบบ real-time หรือ near-real-time

ลักษณะ

- **Unbounded data** — ไม่มีจุดสิ้นสุด
- Event time vs processing time
- เทคโนโลยี: Apache Kafka, Apache Flink, Spark Streaming, Kafka Streams

Stream Processing — ตัวอย่าง



```
# Spark Structured Streaming
stream_df = spark.readStream \
    .format("kafka") \
    .option("subscribe", "transactions") \
    .load()

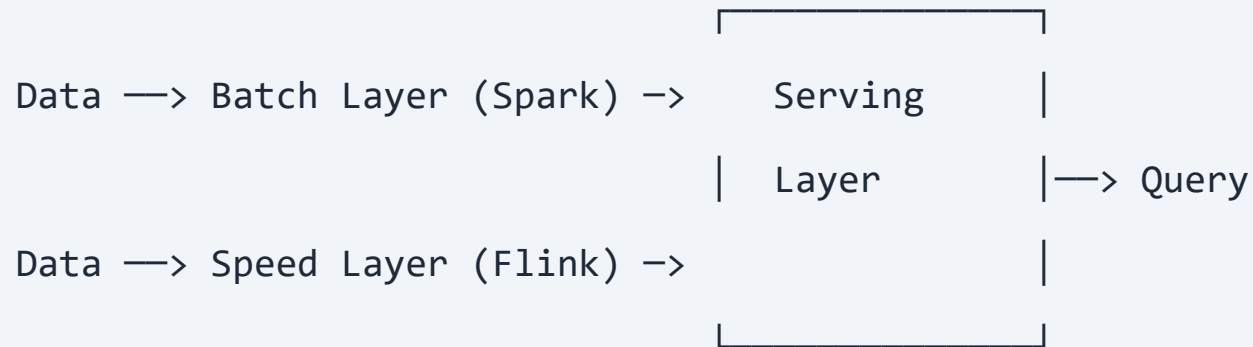
result = stream_df.groupBy("category") \
    .agg(sum("amount").alias("total")) \
    .writeStream \
    .format("console") \
    .outputMode("update") \
    .start()
```

Batch vs Stream Processing

มิติ	Batch Processing	Stream Processing
ข้อมูล	Bounded (finite)	Unbounded (infinite)
เวลา	มีจุดเริ่ม-สิ้นสุด	ต่อเนื่องตลอดเวลา
Latency	นาที ถึง ชั่วโมง	มิลลิวินาที ถึง วินาที
Trigger	Time-based schedule	Event-driven
ความซับซ้อน	ต่ำกว่า	สูงกว่า (state, watermark, late data)
ต้นทุน	ต่ำกว่า	สูงกว่า
Use case	รายงานสรุป, ETL	Alert, dashboard real-time, fraud detection

Lambda Architecture

- **Batch Layer** — ประมวลผลข้อมูลเต็มรูปแบบ (accuracy)
- **Speed Layer** — ประมวลผล real-time (low latency)
- **Serving Layer** — รวบรวมผลลัพธ์จากทั้งสอง layer



Lambda Architecture = ความถูกต้อง (Batch) + ความรวดเร็ว (Stream)

Kappa Architecture

- **แนวคิด:** ใช้ Stream processing เพียงอย่างเดียว ไม่ต้องมี batch layer แยก (ทุกอย่างเป็น stream)
- ข้อมูลย้อนหลัง replay ผ่าน stream processor (seek to offset)

Data —> Kafka —> Stream Processor —> Serving DB —> Query
(Flink / Kafka Streams)

ข้อดี

- สถาปัตยกรรมที่ง่ายกว่า (single stack)
- ไม่ต้อง maintain batch + stream สองระบบ
- Replay ได้ทุกเมื่อ

ข้อเสีย

- ไม่เหมาะกับงาน complex aggregation บางประเภท และ ต้องมีระบบจัดเก็บประวัติข้อมูล (log) ที่ดี

04

การออกแบบ Workflow สำหรับการจัดการ ข้อมูล (ETL / ELT)

การออกแบบ Workflow

- **Workflow** = ลำดับขั้นตอนการทำงานของ data pipeline
- ต้องออกแบบให้ **robust, observable, maintainable**

องค์ประกอบสำคัญ

องค์ประกอบ	คำอธิบาย
Source	ระบบต้นทาง (DB, API, File, Streaming)
Transformation	การแปลงข้อมูล (cleaning, enrichment, aggregation)
Destination	ปลายทาง (Warehouse, Data Lake, API)
Scheduling	การกำหนดเวลา / event trigger
Monitoring	การเฝ้าระวังและแจ้งเตือน
Error Handling	Retry, dead letter queue, logging

Workflow Design — Best Practices

- **Idempotency** — รับซ้ำกี่ครั้งก็ได้ผลลัพธ์เดียวกัน
- **Incremental Loading** — โหลดเฉพาะข้อมูลที่เปลี่ยนแปลง
- **Partitioning** — แบ่งข้อมูลเป็น partition เพื่อประสิทธิภาพ
- **Data Quality Checks** — ตรวจสอบความถูกต้องทุกขั้นตอน
- **Observability** — Logging, monitoring, alerting
- **Backfill Strategy** — รองรับการย้อนประมวลผลข้อมูลเก่า (นบทวน)

ตัวอย่าง Data Pipeline

Source: PostgreSQL (Sales DB)

|

Extract: Sqoop / JDBC (ทุก 6 ชม.)

|

Staging: HDFS / S3 (Raw Format)

|

Transform: Spark (clean, join, aggregate)

|

Load: Data Warehouse

|

Data Mart: Reporting Tables

|

BI Tools: Looker / Tableau / Power BI / Streamlit

ETL — Extract Transform Load

ขั้นตอน

1. **Extract** — ดึงข้อมูลจาก source systems
2. **Transform** — แปลงข้อมูลใน staging area
3. **Load** — โหลดข้อมูลที่ transform แล้วไปยัง destination

```
-- ETL: Transform ก่อน Load  
  
CREATE TABLE dw.sales_summary AS  
  
SELECT  product_id,  
        SUM(qty * price) AS revenue,  
        COUNT(*) AS transactions  
  
FROM    staging.raw_sales  
  
WHERE   status = 'completed'  
  
GROUP BY product_id;
```

ELT — Extract Load Transform

ขั้นตอน

1. **Extract** — ดึงข้อมูลจาก source systems
2. **Load** — โหลด raw data ไปยัง Data Lake/Warehouse โดยตรง
3. **Transform** — แปลงข้อมูลใน target system

```
-- ELT: Load ก่อน, Transform ที่หลัง  
COPY staging.raw_sales FROM 's3://data/sales.csv';  
  
-- ขั้นตอน Transform: แปลงใน Warehouse  
CREATE TABLE mart.daily_revenue AS  
SELECT  date, SUM(revenue) AS total  
FROM    staging.raw_sales  
GROUP BY date;
```

ETL vs ELT

มิติ	ETL	ELT
ลำดับ	Transform ก่อน Load	Load ก่อน Transform
พื้นที่จัดเก็บ	Staging area ชั่วคราว	Raw data ถูกเก็บถาวร
ยืดหยุ่น	ต่ำ — schema fixed	สูง — schema-on-read
ต้นทุน transform	จ่ายก่อนเข้า warehouse	ใช้ compute ของ warehouse
เวลานำ	นานกว่า (transform ก่อน)	เร็วกว่า (load ก่อน แล้วค่อย transform)
ใช้กับ	Data Warehouse แบบดั้งเดิม	Data Lake / Cloud Warehouse
เหมาะกับ	ระบบ legacy, schema ตายตัว	Big Data, ML, data exploration

เครื่องมือจัดการ Workflow

เครื่องมือ	ลักษณะ	จุดเด่น
Apache Airflow	DAG-based scheduler	Python-native, ecosystem ใหญ่
Prefect	Modern Airflow	รองรับ dynamic workflow
Dagster	Data-aware orchestrator	Asset-based, observability สูง
Apache NiFi	Visual data flow	Drag-and-drop, real-time
dbt	Data build tool	Transform in warehouse (ELT)
Kestra	Event-driven	YAML-based, declarative

สรุป Module 01

หัวข้อ	Key Takeaways
Data Engineering	สร้าง infrastructure และ pipeline สำหรับข้อมูล
สถาปัตยกรรมข้อมูล	Data Warehouse → Data Lake → Lakehouse
Batch vs Stream	Batch = ตามเวลา, Stream = real-time, Lambda/Kappa Architecture
ETL vs ELT	ETL = transform ก่อน, ELT = แปลงทีหลังใน warehouse

แนวโน้มปัจจุบัน: ELT + Cloud + Lakehouse + Real-time

คำถาม?

Questions & Discussion

ผศ.ดร. เอกรัฐ รัชกาญจน์

ekarat@as.nida.ac.th

ekaratnida.github.io

คณะสถิติประยุกต์ · สถาบันบัณฑิตพัฒนบริหารศาสตร์ (นิด้า)